

Backend Schema Documentation

Apna Rank

Version: 1.0 **Author:** Founder / Senior Backend Engineer **Status:** Draft **Last Updated:** June 2026

Paired with: [schema.prisma](#) | [TRD v1.0](#)

TABLE OF CONTENTS

1. Schema Overview & Entity Map
2. Table Reference (all tables, purpose, row ownership)
3. Critical Indexes (SQL)
4. Row-Level Security (RLS) Policies
5. Permission Matrix
6. Data Ownership Rules
7. DPDP Compliance Design
8. Minor Data Firewall (Triple-Layer)
9. Encryption & Sensitive Fields
10. Data Retention Policy
11. Developer Notes & Common Queries

1. SCHEMA OVERVIEW & ENTITY MAP

AUTHENTICATION CLUSTER

```

users ————— refresh_tokens
          └── otp_requests
          └── parental consents

```

STUDENT CLUSTER

```

users → student_profiles → attempts → attempt_answers
                                |→ diagnosis_results
                                |→ galti_notebook_entries
                                |→ batch_students

```

QUESTION & TEST CLUSTER

```

questions → test_questions → tests → test_purchases
                                   ↘ attempts

```

└─> live_tests → live_test_registrations
└─> battles → battle_answers

DIAGNOSIS CLUSTER

attempt_answers → mistake_classifications
attempts → diagnosis_results

TEACHER CLUSTER

users → teachers → teacher_earnings → test_purchases

INSTITUTE CLUSTER

users → institutes → batches → batch_students
└─> dpa_records
└─> batch_test_assignments

MISC

notifications
content_reports

2. TABLE REFERENCE

Table	Purpose	Row owned by
<code>users</code>	All users (all roles)	User themselves
<code>refresh_tokens</code>	JWT session management	User themselves
<code>otp_requests</code>	OTP audit + rate-limit	Platform (auth system)
<code>student_profiles</code>	Student-specific profile	Student themselves
<code>parental_consent</code> s	Minor consent audit trail	Platform (DPDP compliance) — immutable
<code>questions</code>	Question bank	Creator (teacher / admin)
<code>tests</code>	Test containers	Creator (teacher / institute / admin)
<code>test_questions</code>	Q-to-test assignments	Test owner
<code>test_purchases</code>	Purchase records	Platform (financial record)
<code>attempts</code>	Student test attempts	Student themselves

Table	Purpose	Row owned by
<code>attempt_answers</code>	Per-question answers	Student themselves
<code>mistake_classifications</code>	Mistake DNA	Platform (auto-generated)
<code>diagnosis_results</code>	Diagnosis summary	Platform (auto-generated, student can read)
<code>galti_notebook_entries</code>	Personal error log	Student themselves
<code>teachers</code>	Teacher profile + verification	Teacher themselves + admin for status
<code>teacher_earnings</code>	Earning ledger	Platform (financial)
<code>institutes</code>	Institute profile	Institute admin
<code>dpa_records</code>	DPA acceptance log	Platform (DPDP compliance) — immutable
<code>batches</code>	Batch management	Institute admin
<code>batch_students</code>	Student-batch membership	Institute admin
<code>batch_test_assignments</code>	Test-to-batch assignments	Institute admin
<code>battles</code>	1v1 battle sessions	Platform (game engine)
<code>battle_answers</code>	Battle answers (from Redis)	Platform (game engine)
<code>live_tests</code>	Scheduled live tests	Platform admin
<code>live_test_registrations</code>	Live test sign-ups	Student themselves
<code>notifications</code>	In-app notifications	Platform (system-generated)
<code>content_reports</code>	Moderation queue	Platform admin

3. CRITICAL INDEXES (SQL)

Prisma generates basic indexes from `@@index()` directives. The following are **additional** indexes to be applied in migration SQL for performance-critical queries.


```

-- =====
-- USERS
-- =====

-- Full-text search on student name (for institute admin search)
CREATE INDEX idx_users_name_trgm ON users
    USING gin (name gin_trgm_ops);

-- Fast lookup for active students
CREATE INDEX idx_users_active_role ON users (role, is_active)
    WHERE is_active = true;

-- =====
-- ATTEMPTS – most-queried table
-- =====

-- Institute dashboard: all attempts for students in an institute
-- Used for batch heatmap and lesson plan computation
CREATE INDEX idx_attempts_student_submitted ON attempts (student_id, submitted_at DESC)
    WHERE status = 'SUBMITTED';

-- IRT recalibration: get all attempts for a question (background job)
CREATE INDEX idx_attempt_answers_question ON attempt_answers (question_id, is_correct)
    WHERE is_correct IS NOT NULL;

-- Minor data firewall: isolate all minor attempts
CREATE INDEX idx_attempts_minor ON attempts (student_id, submitted_at)
    WHERE is_minor_data = true;

-- =====
-- MISTAKE CLASSIFICATIONS – diagnosis engine
-- =====

-- Per-student Mistake DNA aggregation (most frequent mistake type)
CREATE INDEX idx_mistake_student_type ON mistake_classifications
    (student_id, mistake_type, created_at DESC);

-- Per-question difficulty calibration (IRT job)
CREATE INDEX idx_mistake_question ON mistake_classifications (question_id, mistake_type)

-- =====
-- QUESTIONS – question bank performance
-- =====

```

```

-- IRT-based question selection: get questions by difficulty range
CREATE INDEX idx_questions_irt ON questions (exam_category, irt_difficulty)
  WHERE is_active = true AND irt_difficulty IS NOT NULL;

-- Manual-tag fallback (cold start): get questions by exam + difficulty
CREATE INDEX idx_questions_manual ON questions (exam_category, difficulty_tag, subject)
  WHERE is_active = true;

-- =====
-- GALTİ NOTEBOOK – student access patterns
-- =====

-- Student's unresolved mistakes sorted by wrong count (most repeated first)
CREATE INDEX idx_notebook_unresolved ON galti_notebook_entries
  (student_id, wrong_count DESC, created_at DESC)
  WHERE is_resolved = false;

-- Pre-exam reminder: recent + unresolved
CREATE INDEX idx_notebook_recent ON galti_notebook_entries
  (student_id, created_at DESC)
  WHERE is_resolved = false;

-- =====
-- BATTLES – matchmaking
-- =====

-- Matchmaking: find WAITING battles in same exam category
CREATE INDEX idx_battles_matchmaking ON battles (exam_category, status, created_at)
  WHERE status = 'WAITING' AND player2_id IS NULL;

-- =====
-- LIVE TESTS – upcoming test query
-- =====

CREATE INDEX idx_live_tests_upcoming ON live_tests (exam_category, scheduled_at)
  WHERE status IN ('SCHEDULED', 'LOBBY_OPEN');

-- =====
-- TEACHER EARNINGS – payout computation
-- =====

CREATE INDEX idx_earnings_pending ON teacher_earnings (teacher_id, status)
  WHERE status = 'PENDING';

```

```
-- =====  
-- NOTIFICATIONS - unread count (badge)  
-- =====  
  
CREATE INDEX idx_notifications_unread ON notifications (user_id, created_at DESC)  
WHERE is_read = false;
```

4. ROW-LEVEL SECURITY (RLS) POLICIES

Enable RLS on tables that contain sensitive or multi-tenant data.

```
sql
```

```

-- Enable RLS on key tables
ALTER TABLE attempts ENABLE ROW LEVEL SECURITY;
ALTER TABLE attempt_answers ENABLE ROW LEVEL SECURITY;
ALTER TABLE diagnosis_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE galti_notebook_entries ENABLE ROW LEVEL SECURITY;
ALTER TABLE student_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE teacher_earnings ENABLE ROW LEVEL SECURITY;
ALTER TABLE batches ENABLE ROW LEVEL SECURITY;

-- =====
-- POLICY: Student sees only their own attempts
-- =====
CREATE POLICY student_own_attempts ON attempts
  FOR ALL
  USING (student_id = current_setting('app.current_user_id')::uuid);

-- =====
-- POLICY: Institute admin sees only their institute's students
-- =====
CREATE POLICY institute_own_students ON student_profiles
  FOR SELECT
  USING (
    institute_id IN (
      SELECT id FROM institutes
      WHERE contact_user_id = current_setting('app.current_user_id')::uuid
    )
  );

-- =====
-- POLICY: MINOR DATA FIREWALL
-- Institute admins CAN read minor student data (they are the data fiduciary)
-- BUT only if their DPA record exists
-- Platform public queries CANNOT access minor data
-- =====
CREATE POLICY minor_data_firewall ON attempts
  FOR SELECT
  USING (
    -- Adult data: anyone with appropriate role can read their own
    (is_minor_data = false AND student_id = current_setting('app.current_user_id')::
    OR
    -- Minor data: only the student themselves, or institute admin WITH signed DPA
    (is_minor_data = true AND (
      student_id = current_setting('app.current_user_id')::uuid

```



```

    OR
    EXISTS (
        SELECT 1 FROM student_profiles sp
        JOIN institutes i ON i.id = sp.institute_id
        JOIN dpa_records d ON d.institute_id = i.id
        WHERE sp.user_id = attempts.student_id
        AND i.contact_user_id = current_setting('app.current_user_id')::uuid
    )
))
);

-- =====
-- POLICY: Minor data NEVER appears in public leaderboard queries
-- Applied to diagnosis_results for relative ranking
-- =====
CREATE POLICY no_minor_in_public_rank ON diagnosis_results
FOR SELECT
USING (
    -- Public ranking queries: only non-minor data
    (current_setting('app.query_context', true) = 'public_leaderboard'
    AND is_minor_data = false)
OR
    -- Normal queries: standard ownership rules apply
    (current_setting('app.query_context', true) != 'public_leaderboard')
);

-- =====
-- POLICY: Teacher sees only aggregate data on own tests (not individual student PII)
-- Enforced at service layer primarily; RLS as backup
-- =====
CREATE POLICY teacher_aggregate_only ON attempts
FOR SELECT
USING (
    -- Teachers cannot directly query attempts table at all
    -- (service layer returns only aggregated data)
    current_setting('app.current_role', true) != 'TEACHER'
);

```

Implementation note: Set `current_setting` values in Prisma middleware before each request:

typescript

```
// In Prisma middleware:
await prisma.$executeRaw`
  SET LOCAL app.current_user_id = ${userId};
  SET LOCAL app.current_role = ${role};
  SET LOCAL app.query_context = ${queryContext};
`;
```

5. PERMISSION MATRIX

5.1 What each role can do

Operation	Student (Adult)	Student (Minor)	Teacher (Verified)	Institute Admin	Platform Admin
Read own profile	✓	✓	✓	✓	✓
Update own profile	✓	✓	✓	✓	✓
Read own attempts	✓	✓	✗	✗	✓
Read own diagnosis	✓	✓	✗	✗	✓
Browse test marketplace	✓	✓	✓	✓	✓
Purchase test	✓	✗	✗	✗	✗
Attempt test	✓	✓ (institute- assigned only)	✗	✗	✓
Join 1v1 battle	✓	✗	✗	✗	✗
Join live test	✓	✗	✗	✗	✓
See public leaderboard	✓	✗	✗	✗	✓
Use AI doubt solver	✓	✓	✗	✗	✓
Create tests	✗	✗	✓ (verified)	✓ (institute only)	✓

Operation	Student (Adult)	Student (Minor)	Teacher (Verified)	Institute Admin	Platform Admin
Publish to marketplace	✗	✗	✓ (verified)	✗	✓
View own earnings	✗	✗	✓	✗	✓
View aggregate test analytics	✗	✗	✓ (own tests)	✓ (own batches)	✓
View individual student data	✗	✗	✗	✓ (own students)	✓
View batch heatmap	✗	✗	✗	✓ (own batches)	✓
Onboard students	✗	✗	✗	✓	✓
Approve teacher	✗	✗	✗	✗	✓
Remove content	✗	✗	✗	✗	✓

5.2 API Route Guard Implementation

typescript

```
// Role guards applied as Fastify hooks

// Guard: Adult student only (blocks minor)
export const adultStudentGuard = async (request, reply) => {
  if (request.user.isMinor) {
    return reply.status(403).send({
      error: { code: 'MINOR_RESTRICTED', message: 'Yeh feature 18+ ke liye hai' }
    });
  }
};

// Guard: Verified teacher only
export const verifiedTeacherGuard = async (request, reply) => {
  if (request.user.role !== 'TEACHER' || request.user.teacherStatus !== 'VERIFIED')
    return reply.status(403).send({
      error: { code: 'TEACHER_NOT_VERIFIED', message: 'Pehle verification complete k'
    });
  }
};

// Guard: Institute admin only (checks DPA if minor students involved)
export const instituteAdminGuard = async (request, reply) => {
  if (request.user.role !== 'INSTITUTE_ADMIN') {
    return reply.status(403).send({ error: { code: 'FORBIDDEN' } });
  }
  // If accessing minor student data, verify DPA is signed
  if (request.params.includesMinors) {
    const dpa = await prisma.dPARecord.findFirst({
      where: { institute: { contactUserId: request.user.id } }
    });
    if (!dpa) {
      return reply.status(403).send({
        error: { code: 'DPA_REQUIRED', message: 'Minor students ke liye DPA sign kar'
      });
    }
  }
};
```

6. DATA OWNERSHIP RULES

6.1 Core Rule: Who Can Write What

Table	INSERT	UPDATE	DELETE
users	Auth system only (on OTP verify)	User themselves (own row)	Platform admin only
refresh_tokens	Auth system	Auth system (revoke)	Auth system (expiry cleanup)
student_profiles	System (auto on role selection)	Student themselves	Platform admin
parental_consents	Auth system	Auth system only (immutable fields)	Never deleted (audit)
questions	Teacher (verified) / Admin	Creator only (unpublished); Admin always	Admin only (soft-delete via isActive=false)
tests	Teacher / Institute Admin / Admin	Creator only (if unpublished)	Never hard-deleted; isPublished=false only
test_purchases	Payment system only	Payment system only (status updates)	Never deleted (financial record)
attempts	Student (start attempt)	System (auto-save answers, submit)	Never deleted
attempt_answers	System (per answer submit)	System (autosave)	Never deleted
mistake_classifications	System (background job)	System only	Never deleted
diagnosis_results	System (background job)	System only	Never deleted
galti_notebook_entries	System (auto on wrong answer)	Student (add note, mark resolved)	Never hard-deleted

Table	INSERT	UPDATE	DELETE
<code>teachers</code>	Auth system (on teacher registration)	Teacher (profile fields); Admin (status)	Never deleted
<code>teacher_earnings</code>	Payment system	Payment system (status, paidAt)	Never deleted (financial)
<code>institutes</code>	Auth system (on institute registration)	Institute admin (own profile)	Platform admin
<code>dpa_records</code>	Institute admin (on DPA sign)	Never updated (immutable audit)	Never deleted
<code>batches</code>	Institute admin	Institute admin	Institute admin (soft-delete)
<code>battle_answers</code>	Game engine (from Redis flush)	Never	Never
<code>notifications</code>	System	User (isRead=true)	System (cleanup > 90 days)

6.2 Immutability Rules

The following fields, once set, must **never be modified**:

- `parental_consents.approved_at` (consent timestamp is legal evidence)
- `parental_consents.guardian_digi_locker_id`
- `dpa_records.*` (entire table is immutable — DPA is a legal document)
- `test_purchases.razorpay_order_id` / `razorpay_payment_id`
- `teacher_earnings.gross_amount` / `platform_cut` / `teacher_amount` (once paid)
- `users.dob` (after minor status is set)
- `users.is_minor` (after initial assessment — cannot be changed to false)
- `attempts.student_id`, `attempts.test_id` (set on creation, never changed)

Enforce via:

1. Application-layer validation (Zod schema excludes these fields from update DTOs)
2. Postgres triggers for critical financial/legal tables

```
sql

-- Example trigger: prevent DPA modification
CREATE OR REPLACE FUNCTION prevent_dpa_update()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'DPA records are immutable (DPDP compliance). Cannot update.';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER dpa_immutable
BEFORE UPDATE OR DELETE ON dpa_records
FOR EACH ROW EXECUTE FUNCTION prevent_dpa_update();
```

7. DPDP COMPLIANCE DESIGN

7.1 Key Fields Map to DPDP Requirements

DPDP Requirement	Implementation
Verifiable parental consent before processing minor data	<code>parental_consents</code> table; <code>users.is_minor</code> flag; consent flow in auth
Data minimisation	Only necessary fields collected; no behaviour tracking for minors
Purpose limitation	Minor data (<code>isMinorData=true</code>) used only for academic analytics — firewall via RLS
Consent audit trail	<code>parental_consents</code> immutable; <code>dpa_records</code> immutable
Right to erasure	<code>users.is_active=false</code> + scheduled anonymisation job (see Section 10)
Data breach notification (72h)	Sentry alerts; incident response SOP (separate document)
Data Processing Agreement (institute + Apna Rank)	<code>dpa_records</code> table; DPA version tracked
Prohibition on behavioural tracking of children	No <code>is_minor=true</code> users in battle, public leaderboard, IRT cross-user pool

7.2 Minor Data — Fields That Must Never Be Exported or Cross-Joined

The following data paths are **prohibited** for minor users:

- `attempts WHERE is_minor_data = true` → public leaderboard queries
 - `diagnosis_results WHERE is_minor_data = true` → IRT recalibration pool
 - `student_profiles WHERE user.is_minor = true` → marketing, push notifications
 - Any minor `user_id` → battle or live test matchmaking
-

8. MINOR DATA FIREWALL (TRIPLE-LAYER)

Layer 1 — **API Middleware (MinorGuard):**

```
typescript

// Applied on routes that expose public/social features
if (user.isMinor) {
  throw new ForbiddenError('Feature restricted for users under 18');
}
```

Layer 2 — **Service Layer Query Flag:**

```
typescript

// ALL queries that could contribute to cross-user analytics
// explicitly exclude minor data
const attempts = await prisma.attempt.findMany({
  where: {
    examCategory: ExamCategory.SSC_CGL,
    isMinorData: false, // ALWAYS filter minor data out of cross-user queries
    status: 'SUBMITTED'
  }
});
```

Layer 3 — **Postgres RLS Policy:**

```
sql

-- Backup safety net if service layer filter is accidentally removed
-- Policy defined in Section 4 above
```

If all three layers are correctly implemented, minor data cannot leak into:

- Public rank calculations
- IRT difficulty calibration
- Matchmaking pools
- Marketing analytics exports

9. ENCRYPTION & SENSITIVE FIELDS

9.1 Fields Encrypted at Application Layer (before DB insert)

Table	Field	Encryption	Why
teachers	pan_number	AES-256-GCM, per-user key	PAN is sensitive financial identifier
parental_consent	guardian_phone	AES-256-GCM	Guardian's personal data
otp_requests	otp_hash	SHA-256 (one-way)	OTP should never be stored in plain
refresh_tokens	token_hash	SHA-256 (one-way)	Token theft mitigation

9.2 Implementation Pattern

typescript

```

import { createCipheriv, createDecipheriv, randomBytes } from 'crypto';

const ENCRYPTION_KEY = Buffer.from(process.env.FIELD_ENCRYPTION_KEY, 'hex'); // 32 b

export function encryptField(plaintext: string): string {
  const iv = randomBytes(12);
  const cipher = createCipheriv('aes-256-gcm', ENCRYPTION_KEY, iv);
  const encrypted = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final()]);
  const authTag = cipher.getAuthTag();
  // Store as: iv:authTag:ciphertext (all hex)
  return `${iv.toString('hex')}:${authTag.toString('hex')}:${encrypted.toString('hex')}`;
}

export function decryptField(ciphertext: string): string {
  const [ivHex, authTagHex, encryptedHex] = ciphertext.split(':');
  const iv = Buffer.from(ivHex, 'hex');
  const authTag = Buffer.from(authTagHex, 'hex');
  const encrypted = Buffer.from(encryptedHex, 'hex');
  const decipher = createDecipheriv('aes-256-gcm', ENCRYPTION_KEY, iv);
  decipher.setAuthTag(authTag);
  return Buffer.concat([decipher.update(encrypted), decipher.final()]).toString('utf8');
}

```

10. DATA RETENTION POLICY

Data Type	Retention Period	Action on Expiry
Active user accounts	Indefinite (while active)	—
Inactive user accounts	2 years after last login	Anonymise (name → "Deleted User", phone → hash)
OTP requests	30 days	Hard delete
Refresh tokens (expired)	7 days after expiry	Hard delete
Refresh tokens (revoked)	30 days	Hard delete
Minor account (consent not received in 48h)	48 hours	Hard delete entire user record

Data Type	Retention Period	Action on Expiry
Notifications	90 days	Hard delete
Attempt data (active student)	3 years	Archive to cold storage
Financial records (test_purchases, teacher_earnings)	7 years	Never deleted (regulatory)
DPA records	7 years	Never deleted (legal)
Parental consent records	7 years	Never deleted (legal)
Content reports (resolved)	1 year	Hard delete

10.1 Anonymisation on Account Deletion

```

sql

-- Run when user requests deletion (DPDP Right to Erasure)
-- Financial records retained, personal data anonymised

UPDATE users SET
  name = 'Deleted User',
  phone = encode(sha256(phone::bytea), 'hex'), -- one-way hash
  email = NULL,
  dob = NULL,
  is_active = false,
  updated_at = NOW()
WHERE id = $1;

-- Anonymise student profile
UPDATE student_profiles SET
  baseline_level = NULL,
  last_active_at = NULL
WHERE user_id = $1;

-- Anonymise galti notebook (remove personal notes)
UPDATE galti_notebook_entries SET
  student_note = NULL
WHERE student_id = $1;

```

11. DEVELOPER NOTES & COMMON QUERIES

11.1 Setup Commands

```
bash

# Initial setup
npx prisma generate           # generate Prisma client
npx prisma migrate dev       # run migrations (dev)
npx prisma migrate deploy    # run migrations (production)
npx prisma db seed           # seed initial data (exam categories, admin user)
npx prisma studio            # GUI for dev inspection
```

11.2 Environment Variables Required

```
env

# .env
DATABASE_URL="postgresql://USER:PASS@HOST:PORT/DBNAME?pgbouncer=true&connection_limit=10"
DIRECT_DATABASE_URL="postgresql://USER:PASS@HOST:PORT/DBNAME"
FIELD_ENCRYPTION_KEY="64-char-hex-string-generated-with-openssl-rand-hex-32"
```

11.3 Common Query Patterns

```
typescript
```

```

// — Get student diagnosis with Mistake DNA breakdown —————
const diagnosis = await prisma.diagnosisResult.findUnique({
  where: { attemptId },
  include: {
    attempt: {
      include: {
        answers: {
          include: { mistake: true, question: true }
        }
      }
    }
  }
});

// — Batch heatmap: aggregate weak topics for institute batch —————
// NOTE: Always filter isMinorData based on context
const heatmap = await prisma.attemptAnswer.groupBy({
  by: ['questionId'],
  where: {
    attempt: {
      studentProfile: {
        batches: { some: { batchId } }
      },
      status: 'SUBMITTED',
      isMinorData: false // or true if institute-only mode with DPA
    },
    isCorrect: false
  },
  _count: { isCorrect: true },
  orderBy: { _count: { isCorrect: 'desc' } }
});

// — Dropout risk: students inactive > 5 days in a batch —————
const atRisk = await prisma.studentProfile.findMany({
  where: {
    batches: { some: { batchId } },
    lastActiveAt: { lt: new Date(Date.now() - 5 * 24 * 60 * 60 * 1000) }
  },
  include: { user: { select: { name: true, phone: true } } }
});

// — IRT recalibration query (runs in background job) —————
const questionStats = await prisma.attemptAnswer.groupBy({

```

```

by: ['questionId'],
where: {
  questionId,
  isCorrect: { not: null },
  attempt: { isMinorData: false } // exclude minor data from IRT pool
},
_count: { isCorrect: true },
_sum: { isCorrect: true } // Prisma: sum of boolean = count of true
});

// — Teacher aggregate analytics (NO individual student data) —
const testStats = await prisma.attempt.aggregate({
  where: { testId, status: 'SUBMITTED' },
  _avg: { score: true },
  _count: { id: true }
});

// Note: NEVER include individual student data in teacher queries
// Use aggregate + groupBy only

// — Find waiting battle for matchmaking —
const battle = await prisma.battle.findFirst({
  where: {
    examCategory,
    status: 'WAITING',
    player2Id: null,
    player1Id: { not: requestingUserId }, // can't match with yourself
    createdAt: { gt: new Date(Date.now() - 60 * 1000) } // within last 60s
  },
  orderBy: { createdAt: 'asc' } // FIFO matchmaking
});

```

11.4 Prisma Middleware Setup (required for RLS + audit)

typescript

```
// Apply before every Prisma query
prisma.$use(async (params, next) => {
  // Inject current user context for RLS
  if (currentUser) {
    await prisma.$executeRaw`
      SET LOCAL app.current_user_id = ${currentUser.id};
      SET LOCAL app.current_role = ${currentUser.role};
    `;
  }

  // Audit log for writes on sensitive tables
  const sensitiveTables = ['parental_consent', 'dpa_records', 'teacher_earnings'];
  if (sensitiveTables.includes(params.model) &&
    ['create', 'update', 'delete'].includes(params.action)) {
    console.log(`AUDIT: ${params.action} on ${params.model} by ${currentUser?.id}`);
    // In production: write to separate audit_log table or external service
  }

  return next(params);
});
```

11.5 Index Naming Convention

idx_{table}_{column(s)}	- standard index
idx_{table}_{column}_partial	- partial index (WHERE clause)
uq_{table}_{column(s)}	- unique constraint (if not using @@unique)
fk_{table}_{referenced_table}	- foreign key (auto-named by Prisma, rename in migration)

11.6 Migration Checklist (run before each `prisma migrate deploy`)

- ☐ Schema change reviewed for backward compatibility
- ☐ Destructive migration (DROP COLUMN / DROP TABLE) has a rollback plan
- ☐ New indexes added for any new query patterns
- ☐ RLS policies updated if new tables added with sensitive data
- ☐ Minor data isMinorData flag set on any new table joining attempts/students
- ☐ Data retention policy updated if new table created
- ☐ Immutability triggers added for financial/legal tables
- ☐ Performance tested on staging with production-sized dataset

This schema and documentation are the single source of truth for all database decisions. Never modify `parental_consent`s, `dpa_records`, `test_purchases`, or `teacher_earnings` without legal review. Always test RLS policies after any schema migration — policies silently break on table renames.

Document owner: Founder / Backend Lead — Apna Rank **Next review:** After first migration is deployed to staging